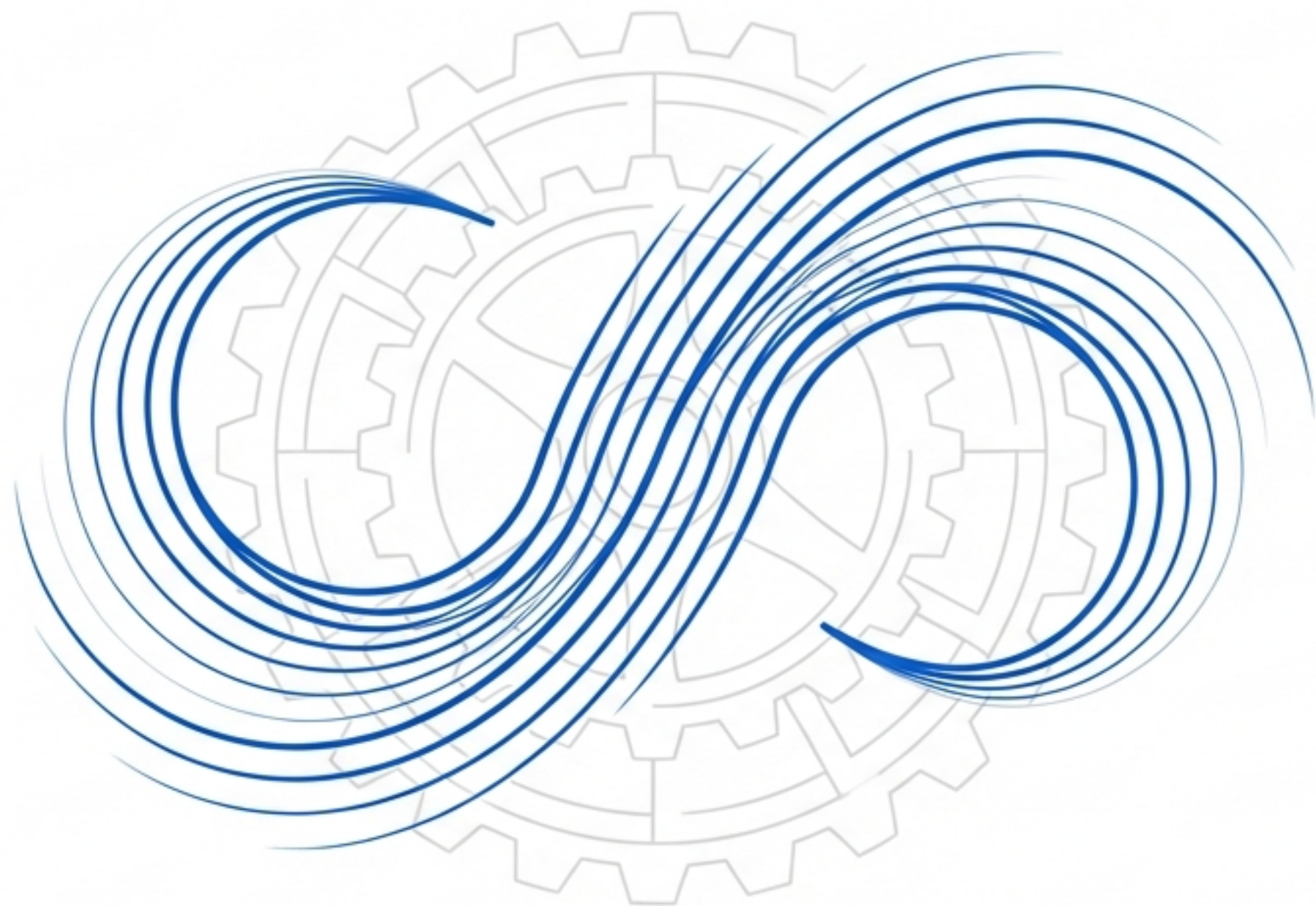




The Orchestrator's Playbook: 次世代AI駆動開発ワークフロー

Claude Code開発者 Boris Cherny氏の実践から学ぶ、生産性を飛躍させる自律的開発システムの構築法

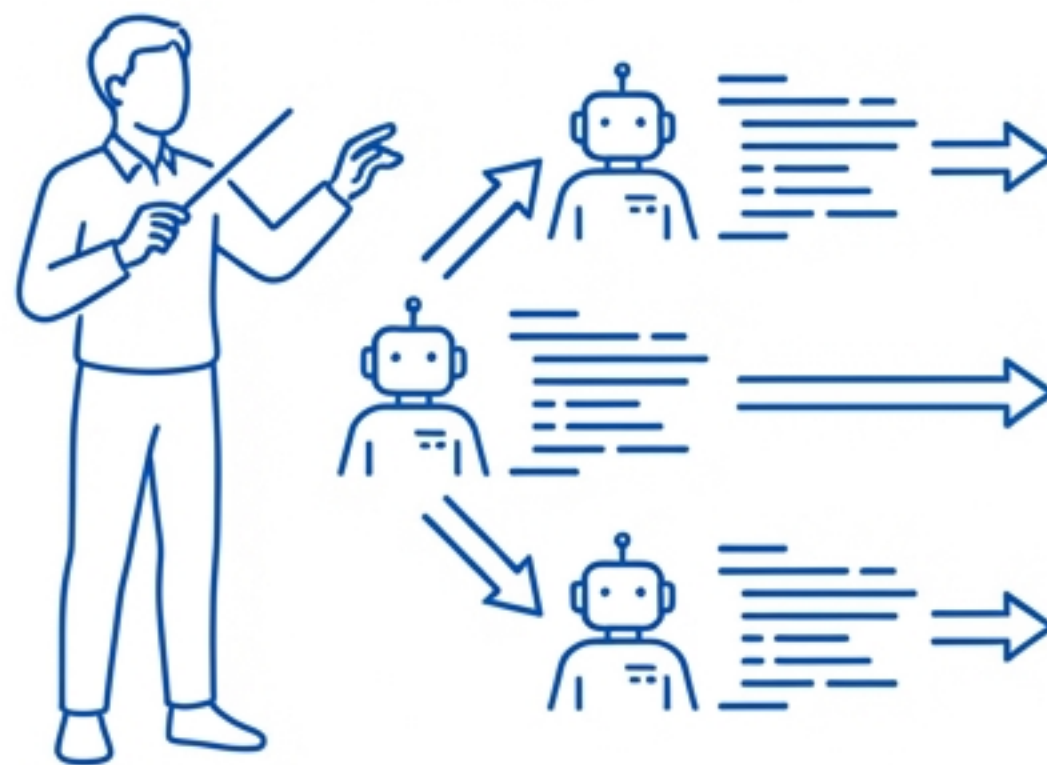
あなたは「コーダー」から「オーケストレーター」へ

Before (コーダー)



- AIを「高速なコード補完ツール」として利用。
- 生成されたコードを一行ずつ手動でレビュー・修正。
- 人間の「ステアリング（修正指示）」がボトルネックとなる。

After (オーケストレーター)



- AIを「自律的な開発エージェントチーム」として指揮。
- AIに自己検証させ、人間は最終成果物と戦略をレビュー。
- 開発プロセス全体を設計・監督する役割へとシフトする。

驚くほど「バニラ」なセットアップに宿る思想



Boris Cherny氏

Head of Claude Code, Anthropic

ツールを過度にカスタマイズしない「バニラ（標準的）」な状態を尊重。

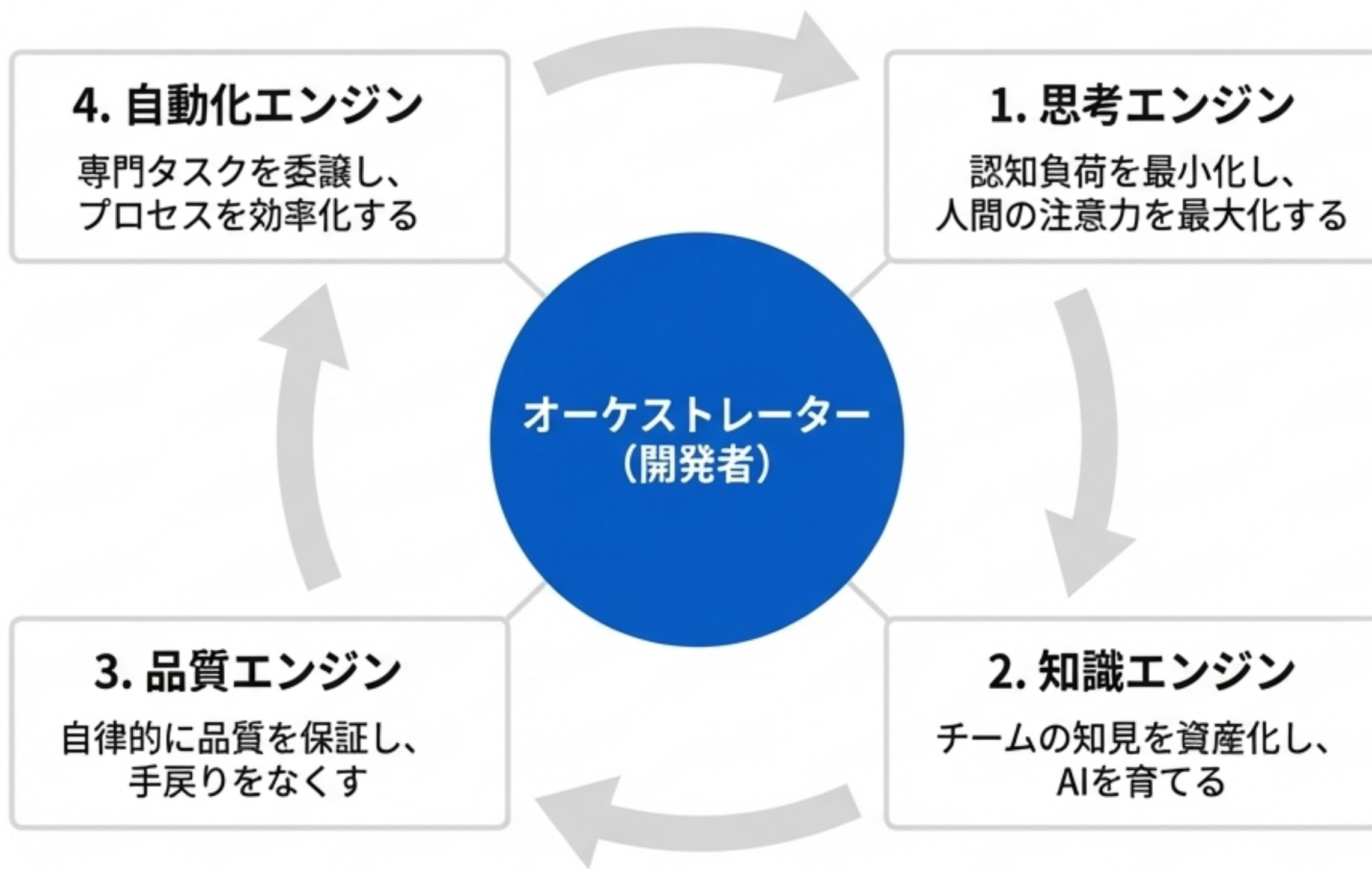
「私の環境は驚くほど『バニラ』かもしれません！」

シンプルな機能を組み合わせ、開発プロセス全体をシステムとして機能させる

「Claude Codeの使い方に唯一の正解はない。
ユーザーが好きなように使い、カスタマイズし、
ハックできるように意図的に設計しています。」

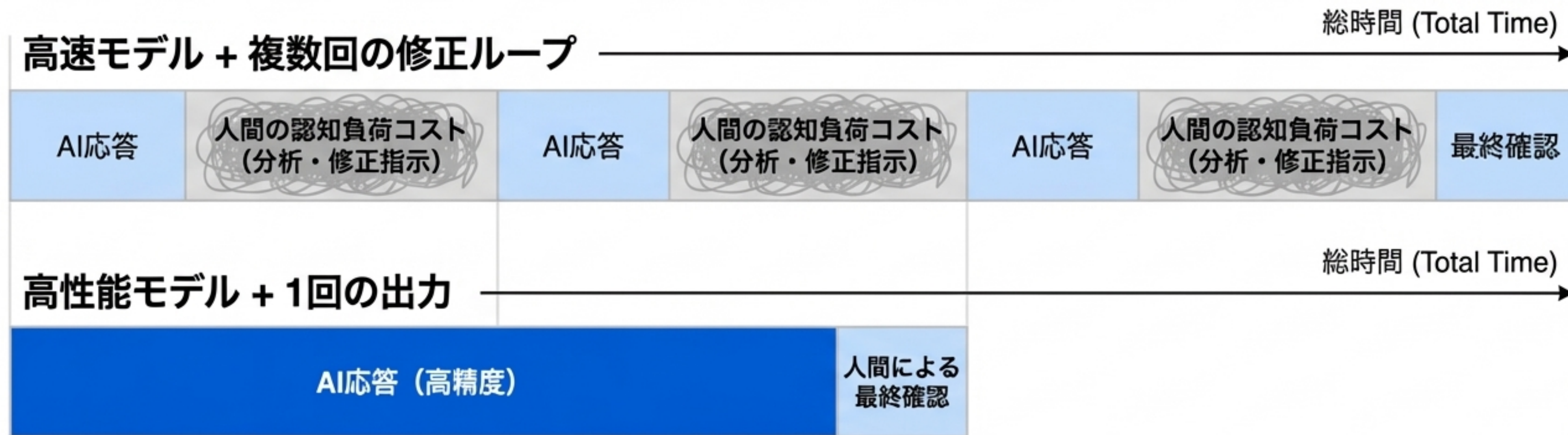
- Boris Cherny

4つのエンジンで駆動する「開発フライホイール」



【思考エンジン①】 「遅いモデルが速い」という逆説

なぜ Opus 4.5（最も高性能なモデル）を使うのか？



- Boris氏は全ての作業で、あえて低速・高価な最上位モデル「Opus 4.5 with thinking」を使用する。
- 理由: 初回での正解率が非常に高いため、人間による修正指示（ステアリング）の回数が劇的に減少する。
- AIの応答を待つ数秒のコストより、「どこが違うか分析し、修正案を考える」という高価な**認知負荷コスト**を支払わない方が、最終的な総時間は短くなる。

【思考エンジン②】 超並列処理で人間を「意思決定」に集中させる

実践方法

- ターミナルで5セッション、Webで5-10セッションを同時実行。
- iTerm2のシステム通知を設定し、AIからの入力要求を即座に把握。
- `--teleport` コマンドでローカルとWebのセッションを自由に行き来する。

なぜこれが可能か？

- 前述の「高性能モデル」が人間の認知負荷を下げているため、複数のセッションを監督・指揮するための「余剰注意力」が生まれる。
- これにより、開発者は常に意思決定だけに集中できる。



【知識エンジン】 複利エンジニアリングとCLAUDE.md

複利エンジニアリングとは？

チームの知見や学びを、時間と共に価値が増大する戦略的資産へと変えるアプローチ。

中核ツール: `CLAUDE.md`

プロジェクトルートに配置する「AI向けオンボーディングドキュメント」。

コード規約、ワークフロー、頻出の落とし穴などを記述。



運用のベストプラクティス



Gitで共有: チームの知識ベースとして育成する。



自動更新: コードレビュー時に`@.claude`をタグ付けし、GitHub Action経由でそのPRの学びを`CLAUDE.md`に追記させる。

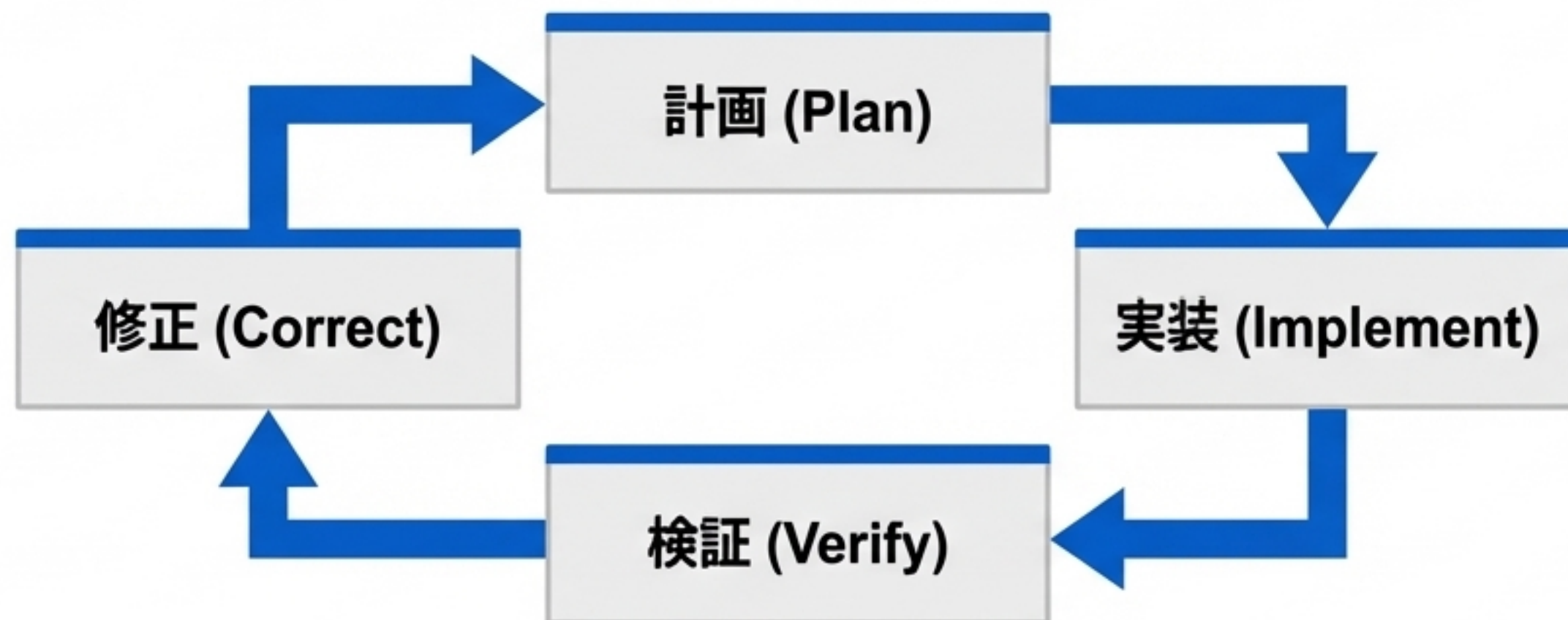


簡潔に保つ: Boris氏は200~300行程度を推奨。過剰なルールは「汚染(pollution)」を招く。

【品質エンジン】最重要戦略：検証ループ

「おそらく Claude Code から最高の結果を得るために最も重要なことは、
『**Claude に自分の仕事を検証させる手段を与えること**』です。
フィードバックループがあれば、最終結果の品質は2〜3倍になります。」

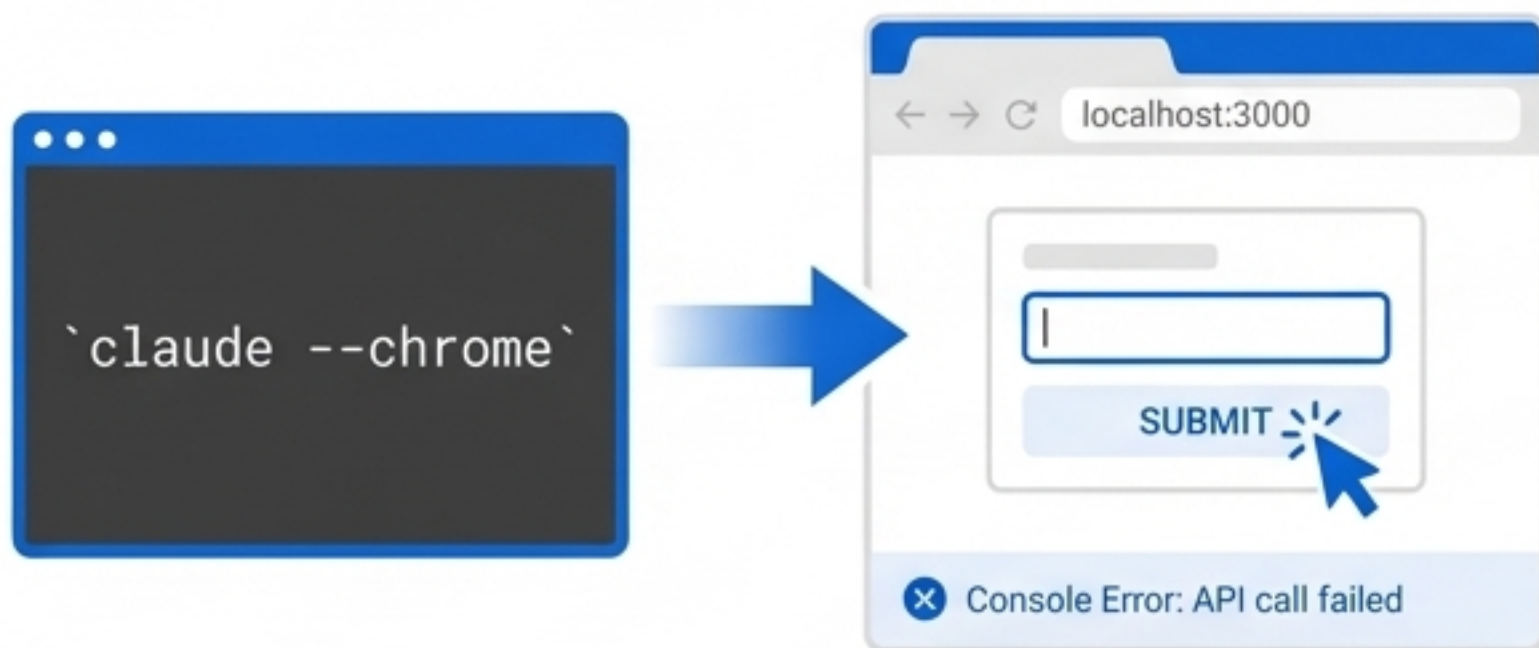
– Boris Cherny



従来: 生成 → 人間が確認 → 修正指示

検証ループ: [計画 → 実装 → 検証 → 修正] のサイクルをAIが自律的に繰り返す。

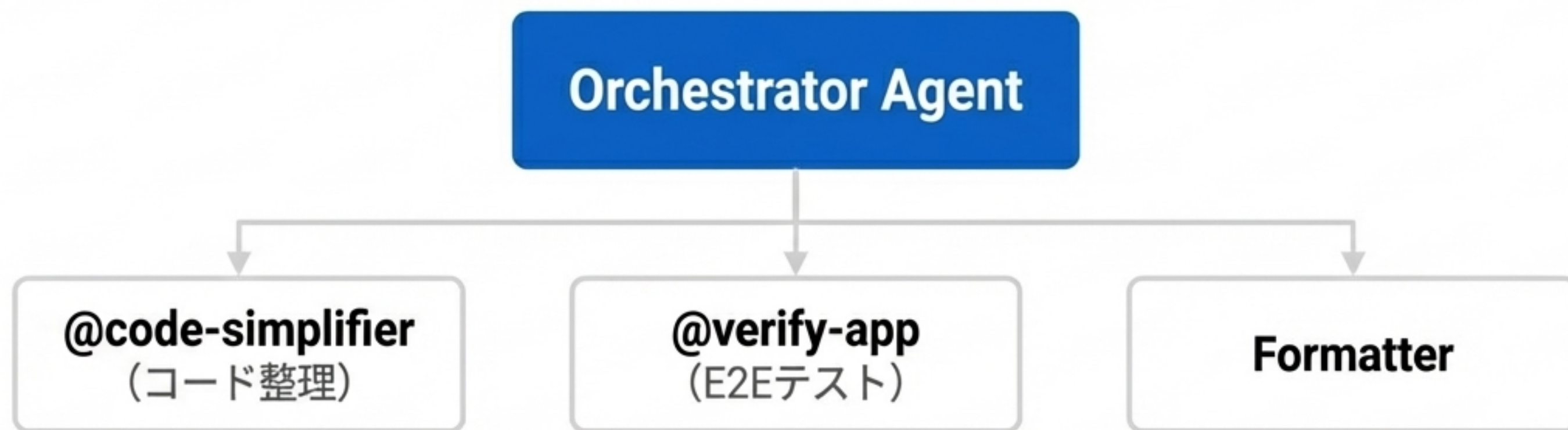
検証ループの実践：Claude Chrome拡張によるUIテスト自動化



主な機能（Key Capabilities）

-  **Webアプリのテスト:** ローカルサーバー（localhost:3000）にアクセスし、フォーム入力やユーザーフローを自動テスト。
-  **ライブデバッグ:** ブラウザのコンソールエラーやDOMを直接読み取り、原因コードを自ら修正。
-  **データ抽出:** Webページから構造化データを抽出し、CSVとして保存。
-  **認証の継承:** ログイン済みのGoogle DocsやNotionなども、ブラウザのCookieを引き継ぐためAPI設定なしで操作可能。

【自動化エンジン】 SubagentとHookによるモジュール化



Subagent (サブエージェント)

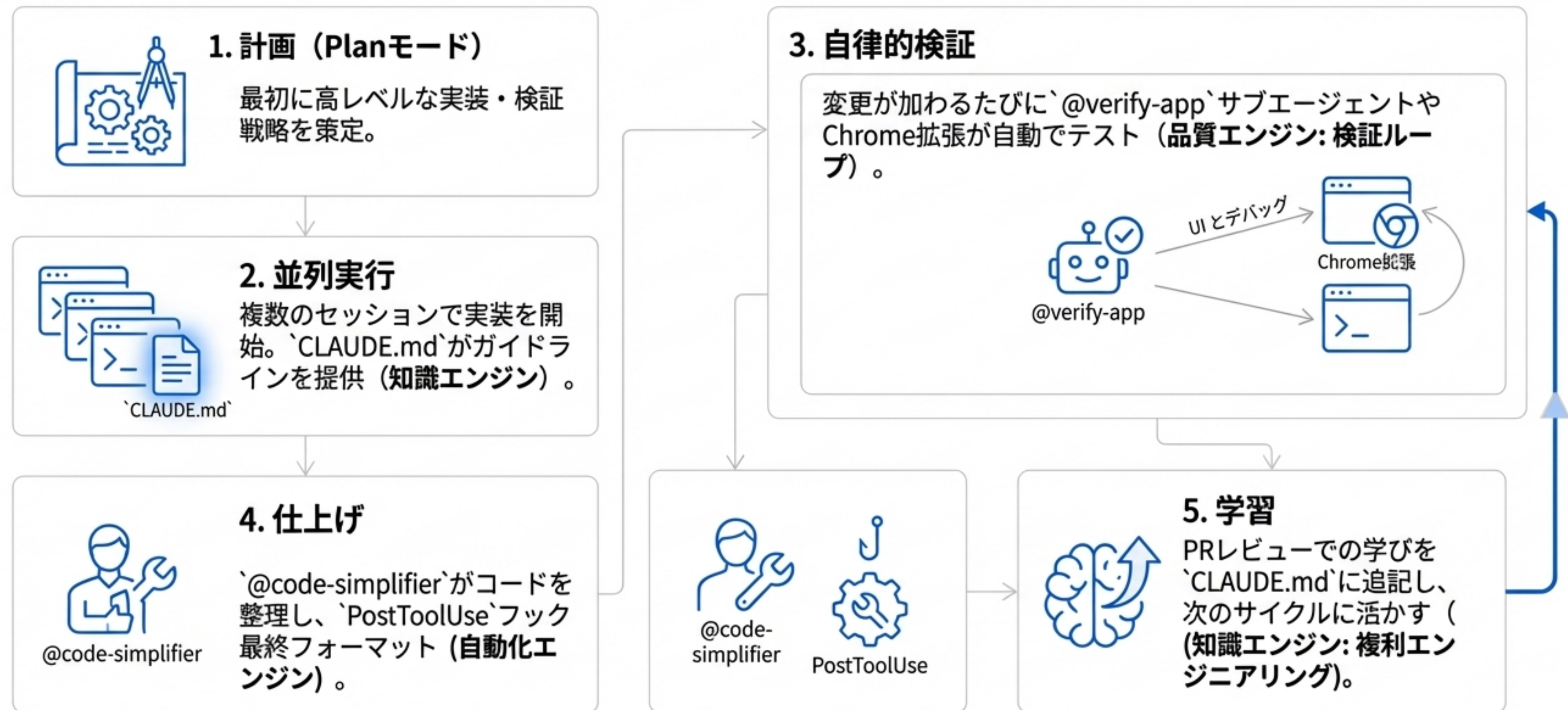
特定のタスク（コード簡素化、テスト実行など）に特化した、再利用可能なClaudeインスタンス。
例: `@code-simplifier` (コード整理), `@verify-app` (E2Eテスト)。
`.claude/agents/` 配下のMDファイルで定義し、チームで共有。親セッションのコンテキストを継承するため軽量にタスクを委譲できる。

Hook (フック)

特定のイベントをトリガーにスクリプトを自動実行する仕組み。
例: `PostToolUse` フックで、コード生成直後に自動でフォーマッターを実行し、CIエラーを未然に防止。

オーケストレーター的一天：すべてが連携する実践フロー

これは、知識、自動化、品質エンジンの各コンポーネントが統合された実践的な開発ワークフローの例です。



コードを書く者から、価値を創造するシステムを設計する者へ

Boris Cherny氏のワークフローは、単なる生産性向上術ではない。
それは、開発者の役割そのものを再定義する**次世代の開発パラダイム**である。

AIに自己修正能力を与える **「検証ループ」**

チームの知識を資産化する **「複利エンジニアリング」**

人間の認知負荷を最小化する **「ワークフローのオーケストレーション」**

これらを組み合わせることで、私たちはより創造的で、より高次元の課題解決に集中できる。

明日から始める「オーケストレーター」への第一歩

1



Step 1 (今日から):

最も高性能なモデル (Opus 4.5) を試す。修正指示（ステアリング）の回数がどう変化するかを体感する。

2



Step 2 (今週中に):

既存プロジェクトに`CLAUDE.md`を導入し、最も重要なルールを3つだけ記述してみる。

3



Step 3 (今月中に):

あなたのプロジェクトで最も簡単な「検証ループ」を一つ構築する（例: Claudeに`npm test`を実行させる、Chrome拡張でトップページを開かせる）。

主な情報源 (Sources)

Boris Cherny氏のX (旧Twitter) スレッド (2026年1月2日投稿)

<https://x.com/bcherny/status/2007179832300581177>

Claude Code 公式ドキュメント

- Terminal configuration: [[docs/en/terminal-config](#)]
- Use Claude Code with Chrome (beta): [[docs/en/chrome](#)]
- Subagents: [[docs/en/sub-agents](#)]
- Hooks: [[docs/en/hooks-guide](#)]

関連する解説・翻訳記事

- 篠塚孝哉氏によるXでの和訳・まとめ (2026年1月3日投稿)
- <https://x.com/shinojapan/status/2007254196077375970>